



UNIVERSIDAD SERGIO ARBOLEDA

INFORME TALLER ALGORITMOS GENÉTICOS

INTELIGENCIA ARTIFICIAL

DOCENTE

JOAQUÍN FERNANDO SÁNCHEZ CIFUENTES

GABRIELA DELGADO TIBAMOSO

JULIAN CAMILO CARO HERRERA

NICOLÁS STEVEN PÉREZ CHUSCANO

1 Formulación del problema

Sea $x \in \{0, 1\}^{10}$ con $x_i = 1$ si el proyecto P_i se selecciona. Con $c = (12, 7, 11, 8, 9, 14, 6, 10, 5, 13)$ y $b = (24, 13, 23, 15, 16, 28, 11, 19, 9, 25)$, el problema es

$$\max_{x \in \{0,1\}^{10}} B(x) = \sum_{i=1}^{10} b_i x_i \quad \text{sujeto a} \quad C(x) = \sum_{i=1}^{10} c_i x_i \leq W = 50. \quad (1)$$

Variable de decisión. El vector binario x ; cada x_i es una variable binaria que indica la inclusión del proyecto P_i .

Función objetivo. $B(x)$, el beneficio total, a maximizar.

Restricción principal. $C(x) \leq 50$. Es la única restricción; la condición de que cada proyecto se elija a lo sumo una vez está ya contenida en la codificación binaria y no requiere restricción adicional.

Representación del individuo. Cromosoma binario de diez genes, un gen por proyecto. La correspondencia entre gen y proyecto es posicional y fija.

Tamaño del espacio de búsqueda. $|\Omega| = 2^{10} = 1024$ soluciones (1023 si se excluye la selección vacía). De ellas, 587 satisfacen la restricción, es decir el 57,3%.

Evaluación del individuo solicitado. Para $X = (1, 0, 1, 1, 0, 0, 1, 0, 1, 0)$, que selecciona $\{P_1, P_3, P_4, P_7, P_9\}$:

$$C(X) = 12 + 11 + 8 + 6 + 5 = 42, \quad B(X) = 24 + 23 + 15 + 11 + 9 = 82.$$

Como $42 \leq 50$, la solución es **válida**. No es óptima: dista 18 unidades de beneficio del óptimo global.

2 Caracterización de la instancia

Esta instancia tiene tres propiedades que condicionan cualquier lectura de los resultados. Conviene fijarlas antes de medir nada.

Es resoluble de forma exacta. Con $|\Omega| = 1024$, la enumeración exhaustiva termina en milisegundos, y la programación dinámica clásica de la mochila lo hace en $O(nW) = 500$ operaciones. Los dos métodos se implementaron por separado y coinciden:

$$B^* = 100, \quad C = 50 \text{ exacto}, \quad X^* = 1010010001 = \{P_1, P_3, P_6, P_{10}\},$$

y el óptimo es *único*. Tener el óptimo de antemano cambia la pregunta: ya no es “¿qué solución encontró el algoritmo?” sino “¿con qué frecuencia y a qué costo encuentra la óptima?”, que sí se puede medir.

La heurística voraz la resuelve óptimamente. Ordenando los proyectos por razón b_i/c_i decreciente y tomando cada uno mientras quepa en el presupuesto se obtiene $P_3 \rightarrow P_1 \rightarrow P_6 \rightarrow P_{10}$, con costo exactamente 50 y beneficio 100: el óptimo global, en diez operaciones, sin aleatoriedad y sin parámetros que ajustar. El voraz no garantiza optimalidad en la mochila 0-1, así que esto es un hecho sobre esta instancia y no un teorema. Aun así, condiciona todo lo que venga después.

El paisaje es plano. Las razones b_i/c_i están comprimidas en el intervalo $[1,778, 2,091]$: todos los proyectos son de eficiencia casi idéntica. Cualquier selección que agote el presupuesto queda cerca del óptimo: hay 12 soluciones a lo sumo 3 unidades por debajo de B^* . De ahí una consecuencia para la medición: *el beneficio medio no distingue una configuración de otra*. En los experimentos de este trabajo varía entre 94,98 y 100,00 mientras la tasa de éxito varía entre 0,04 y 1,00. Por eso la métrica primaria se fijó, antes de observar dato alguno, como la fracción de ejecuciones que alcanzan B^* exactamente.

3 Función de aptitud y coeficiente de penalización

Se adopta la penalización lineal exterior

$$f_\lambda(x) = B(x) - \lambda \max(0, C(x) - W), \quad (2)$$

que deja intacta la región factible y degrada linealmente la infactible. La pregunta no es qué valor de λ “funciona”, sino a partir de qué valor el óptimo del problema penalizado coincide con el del original.

3.1 Una cota de uso habitual que es falsa

Se suele afirmar que basta tomar $\lambda \geq \max_i b_i/c_i$, por analogía con la relajación fraccionaria. No es cierto. Tómese una instancia con un único objeto de costo 100 y beneficio 200 (razón 2) y presupuesto $W = 50$. Con $\lambda = 2$, la solución infactible que toma el objeto vale $200 - 2 \cdot 50 = 100$, mientras que la única solución factible — el conjunto vacío — vale 0. El óptimo del problema penalizado es infactible pese a cumplirse la cota. Que en nuestra instancia $\lambda = 2,09$ baste es una casualidad de estos datos, no una consecuencia de la cota.

3.2 Una cota suficiente correcta

Proposición 1. *Si los costos c_i son enteros positivos y $\lambda \geq \max_i b_i$, entonces $\max_x f_\lambda(x) = B^*$, y el máximo se alcanza en una solución factible.*

Demostración. Sea S infactible, con exceso $E = C(S) - W \geq 1$ (entero, por serlo los costos). Al retirar un proyecto $i \in S$, la aptitud varía en

$$\Delta f = -b_i + \lambda \min(E, c_i).$$

Como $c_i \geq 1$ y $E \geq 1$, se tiene $\min(E, c_i) \geq 1$, luego $\Delta f \geq \lambda - b_i \geq 0$. Retirar proyectos nunca empeora la aptitud, y tras un número finito de retiradas se alcanza una solución factible, cuya aptitud coincide con su beneficio. Por tanto ninguna solución infactible supera a la mejor factible. $\square$

Para esta instancia, $\max_i b_i = 28$.

3.3 El umbral exacto de la instancia

La cota anterior es válida en general y, por ello, holgada. El umbral exacto se obtiene de la condición de que ninguna solución infactible supere a B^* :

$$\lambda^* = \max_{x: C(x) > W} \frac{B(x) - B^*}{C(x) - W} = 1,9, \quad (3)$$

calculado por enumeración sobre las 437 soluciones infactibles. El máximo se alcanza en $\{P_1, P_3, P_6, P_8, P_{10}\}$, con $B = 119$ y $C = 60$:

$$f_{1,9} = 119 - 1,9 \cdot 10 = 100 = B^*.$$

En $\lambda = \lambda^*$ hay, por tanto, *empate exacto* de aptitud entre el óptimo global y esa solución infactible. Obsérvese que esa solución es $X^* \cup \{P_8\}$: difiere del óptimo en un solo gen. Esta observación, hecha antes de ejecutar el algoritmo, produce una predicción falsable que la Sección 6.2 contrasta.

Se dispone así de tres valores de referencia: $\lambda^* = 1,9$ (exacto, específico de la instancia), $\lambda \geq 28$ (suficiente, general y demostrable) y $\lambda = 5$, el valor sugerido, que queda entre ambos sin justificación asociada.

4 Operadores

Se implementan los operadores solicitados; las decisiones no triviales son tres.

El torneo no consume presupuesto de evaluaciones. La población se evalúa una vez por generación y la selección por torneo opera sobre las aptitudes ya calculadas. Si el torneo recalculase la aptitud, el número real de evaluaciones excedería $N \times G$ y las comparaciones a presupuesto igualado de la Sección 5 quedarían invalidadas sin señal visible. Se verifica automáticamente que el conteo es exactamente $N \times G$.

Punto de cruce en $\{1, \dots, n-1\}$. Los cortes en 0 y en n producen descendientes idénticos a los padres; admitirlos haría que $p_c = 0,80$ dejase de medir recombinación efectiva.

Mutación por gen. Cada gen se invierte de forma independiente con probabilidad p_m , de modo que el número esperado de genes mutados por individuo es $n p_m$: 0,1 para $p_m = 0,01$, 0,5 para 0,05 y 1,0 para 0,10. Con $p_m = 0,01$, el 90 % de los individuos pasa intacto a la siguiente generación.

4.1 Traza completa de una iteración

La traza no está construida a mano: es una iteración real del programa con semilla 26, reproducible ejecutando `traza_manual.py`. La semilla 42 sugerida en el material produce una traza degenerada — los dos padres comparten el sufijo, el cruce devuelve copias idénticas y no muta ningún gen —, inservible como ilustración; se eligió 26 entre las primeras 500 por cumplir tres criterios fijados de antemano: cruce con efecto sobre ambos descendientes, exactamente una mutación y un padre factible frente a otro infactible.

	Cromosoma	C	B	f_5	
Padre 1 (torneo entre 14, 7, 8)	1101010000	41	80	80,0	factible
Padre 2 (torneo entre 4, 3, 9)	1011110000	54	106	86,0	infactible
<i>Cruce de un punto, sorteo $0,7635 < 0,80$, punto de corte $k = 4$</i>					
Hijo 1 (antes de mutar)	1101 110000	50	96	96,0	factible
Hijo 2 (antes de mutar)	1011 010000	45	90	90,0	factible
<i>Mutación, $p_m = 0,05$: se invierte el gen 6 del hijo 1</i>					
Hijo 1 (final)	1101100000	36	68	68,0	factible
Hijo 2 (final)	1011010000	45	90	90,0	factible

La traza ilustra dos hechos que el análisis posterior retoma. Primero, la penalización opera: el padre 2 tiene el mayor beneficio bruto de los dos (106) y aun así compite con aptitud 86 por violar el presupuesto en cuatro unidades. Segundo, el cruce produce un descendiente ($f = 96$) *mejor que ambos padres*, combinando el prefijo de uno con el sufijo del otro, mientras que la mutación posterior degrada ese mismo descendiente de 96 a 68. La mutación es una perturbación aleatoria, no un operador de mejora, y esa asimetría es la razón de ser del elitismo.

5 Metodología experimental

El diseño experimental sugerido en el enunciado — configuraciones A, B y C — no permite responder las preguntas que él mismo plantea, por dos razones independientes.

Los factores están confundidos. A, B y C modifican simultáneamente el tamaño de población, el número de generaciones y la tasa de mutación. Ningún efecto observado puede atribuirse a un factor concreto.

El presupuesto de cómputo no está igualado. El número de evaluaciones de la función de aptitud es $N \times G$:

Configuración	$N \times G$	Evaluaciones	Veces $ \Omega $
A	10×50	500	0,5
B	20×100	2 000	2,0
C	50×200	10 000	9,8

La configuración C dispone de veinte veces el presupuesto de A y evalúa el equivalente a casi diez veces el espacio de búsqueda completo. Cualquier comparación entre ambas mide sobre todo cómputo, no calidad algorítmica.

Correcciones adoptadas. Se ejecutan A, B y C tal como se definen, para responder al enunciado literalmente, y además un diseño factorial 3×3 sobre $N \in \{10, 20, 50\}$ y $p_m \in \{0,01, 0,05, 0,10\}$ con **presupuesto fijo de 2 000 evaluaciones**, despejando $G = 2000/N$. Cada configuración se ejecuta con las **mismas 100 semillas** (diseño pareado). El número de semillas se fijó por precisión, no por conveniencia: el semiancho del intervalo de Wilson al 95 % en torno a $p \approx 0,9$ es de unos ± 11 puntos con 30 semillas y ± 6 con 100; con 30 semillas, tasas del 85 % y del 95 % serían indistinguibles. El costo computacional no es la restricción: el plan completo (2 200 ejecuciones) tarda 42 segundos.

Métricas. Primaria: tasa de éxito, la fracción de ejecuciones que alcanzan B^* , con intervalo de Wilson — y no la aproximación normal, que con p próximo a 1 produce límites fuera de $[0, 1]$ y subestima la incertidumbre. Secundarias: generación del primer acierto, tratada como variable censurada a la derecha (se reporta la mediana de los casos observados y la fracción censurada, nunca la media, que estaría sesgada); tasa de infactibilidad de la solución devuelta; y diversidad poblacional, medida como número de individuos distintos y distancia de Hamming media al mejor individuo.

Línea base. Búsqueda aleatoria uniforme sobre $\{0, 1\}^{10}$ con el mismo presupuesto de evaluaciones. Su probabilidad de éxito admite forma cerrada,

$$P(\text{éxito} \mid k) = 1 - \left(1 - \frac{1}{1024}\right)^k, \quad (4)$$

que da 38,6 % para $k = 500$, 85,8 % para $k = 2000$ y 99,994 % para $k = 10\,000$. La expresión analítica sirve además como prueba del arnés experimental: la tasa empírica simulada cae dentro del intervalo de Wilson de la predicción en los tres presupuestos, lo que verifica el código de medición antes de aplicarlo al algoritmo genético.

Contraste. Al ser un diseño pareado, la comparación entre el algoritmo genético y la línea base emplea la prueba exacta de McNemar sobre los pares discordantes, con corrección de Holm–Bonferroni por las doce comparaciones simultáneas.

6 Resultados

6.1 Efecto de los parámetros a presupuesto igualado

Cuadro 1: Tasa de éxito con presupuesto fijo de 2 000 evaluaciones (100 semillas; intervalo de Wilson al 95 %).

	$p_m = 0,01$	$p_m = 0,05$	$p_m = 0,10$
$N = 10$	0,05 [0,02–0,11]	0,18 [0,12–0,27]	0,75 [0,66–0,82]
$N = 20$	0,11 [0,06–0,19]	0,31 [0,23–0,41]	0,88 [0,80–0,93]
$N = 50$	0,29 [0,21–0,39]	0,62 [0,52–0,71]	0,90 [0,83–0,94]

Con el cómputo igualado (Tabla 1, Figura 1), la tasa de mutación domina: pasar de $p_m = 0,01$ a 0,10 multiplica la tasa de éxito por un factor de 8 a 15, mientras que quintuplicar la población la multiplica por entre 1,2 y 6. El efecto de la población es además decreciente: entre $N = 20$ y $N = 50$ con $p_m = 0,10$ la diferencia es de dos puntos, dentro del ruido.

Hay un dato que desmonta el aparente éxito con mutación baja: con $p_m = 0,01$ la mediana de la generación del primer acierto es de 0 a 3, con censura del 71 al 95 %. Es decir, las pocas ejecuciones exitosas lo son porque el óptimo ya estaba en la población inicial. Con mutación baja, el algoritmo no encuentra el óptimo: lo hereda del sorteo inicial.

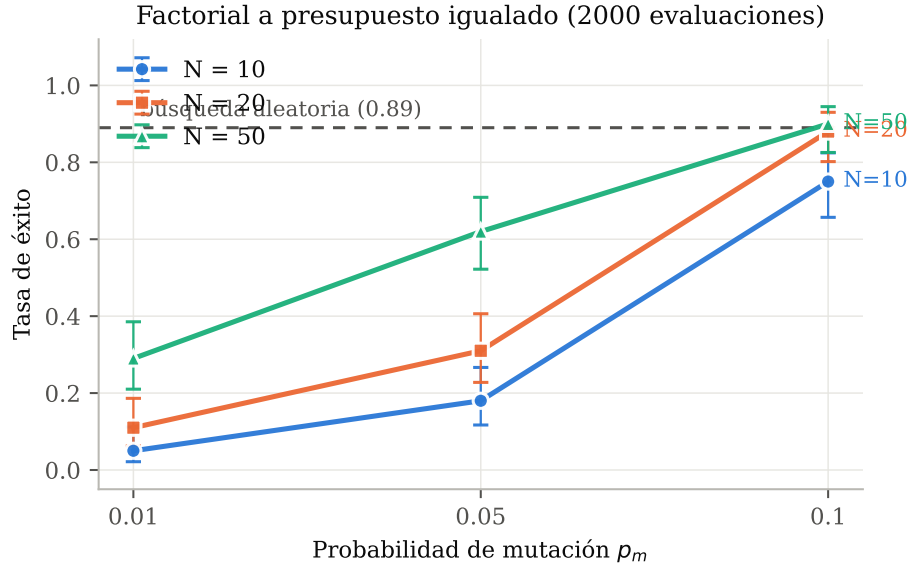


Figura 1: Interacción entre población y mutación a presupuesto igualado. La línea discontinua marca la línea base de búsqueda aleatoria con el mismo presupuesto.

6.2 El coeficiente de penalización

Cuadro 2: Barrido de λ con $N = 20$, $G = 100$, $p_m = 0,05$, 100 semillas.

λ	0	0,5	1,0	1,5	1,9	2,1	3,0	5,0	10	28
Tasa de éxito	0,06	0,07	0,07	0,07	1,00	0,65	0,38	0,31	0,24	0,27
Infactibles	1,00	1,00	1,00	1,00	0,62	0,01	0,00	0,00	0,00	0,00

La predicción derivada en la ecuación (3) se cumple: la transición entre devolver soluciones

infactibles y factibles ocurre exactamente en $\lambda^* = 1,9$, y en ese punto el comportamiento es ambivalente (62 de 100 ejecuciones infactibles), como corresponde a un empate exacto de aptitud en el que el algoritmo carece de criterio para preferir una u otra.

El mecanismo detrás de cada régimen se midió registrando el atractor dominante, es decir el individuo de máxima aptitud al que converge la población:

λ	Atractor dominante	Hamming a X^*	Frecuencia
1,5	111111111 (todos los proyectos)	6	100/100
1,9	$\{P_1, P_3, P_6, P_8, P_{10}\} / X^*$	1 / 0	62 / 38
2,1	X^*	0	65/100
5,0	X^* , resto disperso a Hamming 3, 5 y 6	0	31/100

Con λ por debajo del umbral la restricción se ignora por completo y el algoritmo converge, en las cien ejecuciones, a “seleccionar todos los proyectos”. Con λ en el umbral, la región infactible actúa como *punto*: el atractor es $X^* \cup \{P_8\}$, a una única mutación del óptimo, y basta un cambio de bit para alcanzarlo; de ahí la tasa del 100 %. Con λ grande la región infactible se convierte en un muro que compartimenta el espacio, y la búsqueda queda atrapada en óptimos locales factibles situados a distancia 3, 5 y 6 del óptimo.

Dos cosas se siguen de aquí. Una: el valor $\lambda = 5$ del enunciado rinde 0,31 frente a 0,65 de $\lambda = 2,1$, esto es, menos de la mitad. El parámetro con mayor efecto sobre el desempeño es precisamente el único que el enunciado fija sin justificar y no propone variar. Y otra, por honestidad: el 1,00 de $\lambda = 1,9$ es un artefacto de filo de navaja que depende del empate exacto y, por tanto, de conocer B^* de antemano. No es una recomendación práctica; lo valioso es el mecanismo que revela. La recomendación defendible es situar λ ligeramente por encima de λ^* .

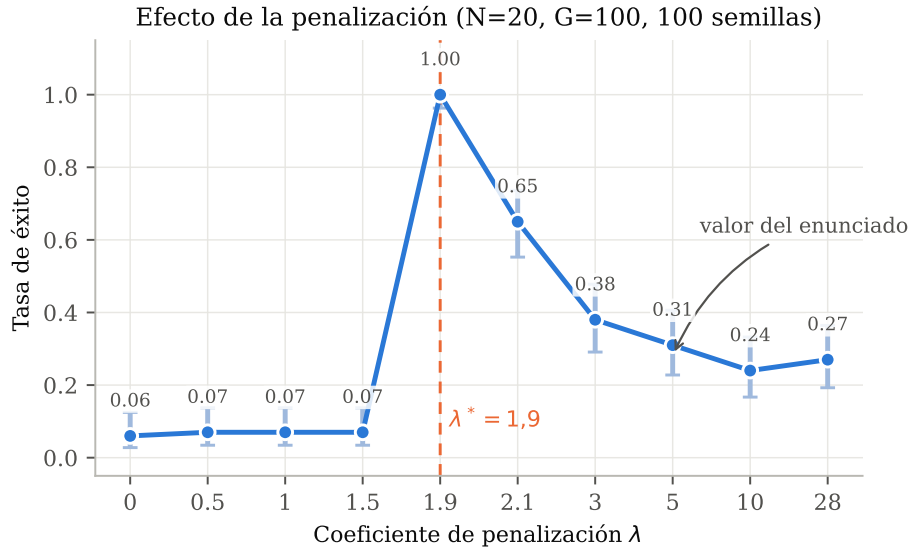


Figura 2: Tasa de éxito frente al coeficiente de penalización, con intervalos de Wilson al 95 %.

6.3 Comparación con la línea base

La Tabla 3 recoge las comparaciones extremas de las doce realizadas (Figura 3). El algoritmo genético **no supera significativamente a la búsqueda aleatoria en ninguna configuración**. Empata en tres y es significativamente peor en ocho, con diferencias de hasta $-0,84$.

Las tres configuraciones que empatan son exactamente las de $p_m = 0,10$, que corresponde a $np_m = 1$ gen mutado por individuo en promedio: el régimen en que el algoritmo genético

Cuadro 3: Algoritmo genético frente a búsqueda aleatoria con idéntico presupuesto de evaluaciones. Prueba exacta de McNemar sobre pares discordantes, con corrección de Holm por doce comparaciones.

Configuración	Diferencia	Solo AG	Solo azar	p (Holm)
$N = 50, p_m = 0,10$	+0,01	11	10	1,000
C (enunciado)	0,00	0	0	1,000
$N = 20, p_m = 0,10$	-0,01	9	10	1,000
$N = 10, p_m = 0,10$	-0,14	8	22	0,065
$N = 50, p_m = 0,05$	-0,27	6	33	0,0001
A (enunciado)	-0,38	2	40	< 0,0001
B (enunciado)	-0,58	4	62	< 0,0001
$N = 10, p_m = 0,01$	-0,84	1	85	< 0,0001

más se aproxima a un muestreo aleatorio. El algoritmo alcanza a la línea base cuando deja de comportarse como un algoritmo genético.

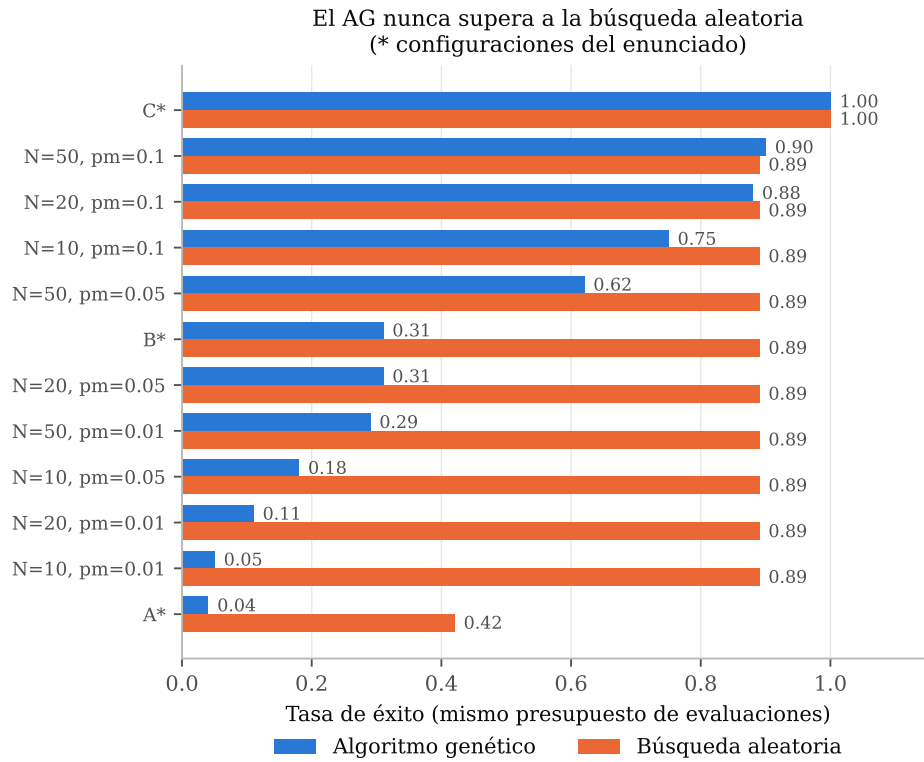


Figura 3: Tasa de éxito de cada configuración frente a la búsqueda aleatoria con el mismo presupuesto de evaluaciones.

6.4 Convergencia prematura

La causa es la pérdida de diversidad. Con $N = 20$, el número de individuos distintos cae de 19,9 a 9,2 en diez generaciones y se estabiliza en torno a 7,8; la distancia de Hamming media al mejor individuo cae de 4,70 a 0,56 (Figura 4). A partir de la generación 20 la población es esencialmente un conjunto de clones del elite.

Ahí está la asimetría con la línea base: las 2000 evaluaciones del algoritmo genético no son 2000 muestras distintas, sino unas pocas centenas de individuos distintos reevaluados una y otra vez, mientras que las 2000 de la búsqueda aleatoria sí son muestras independientes. En un

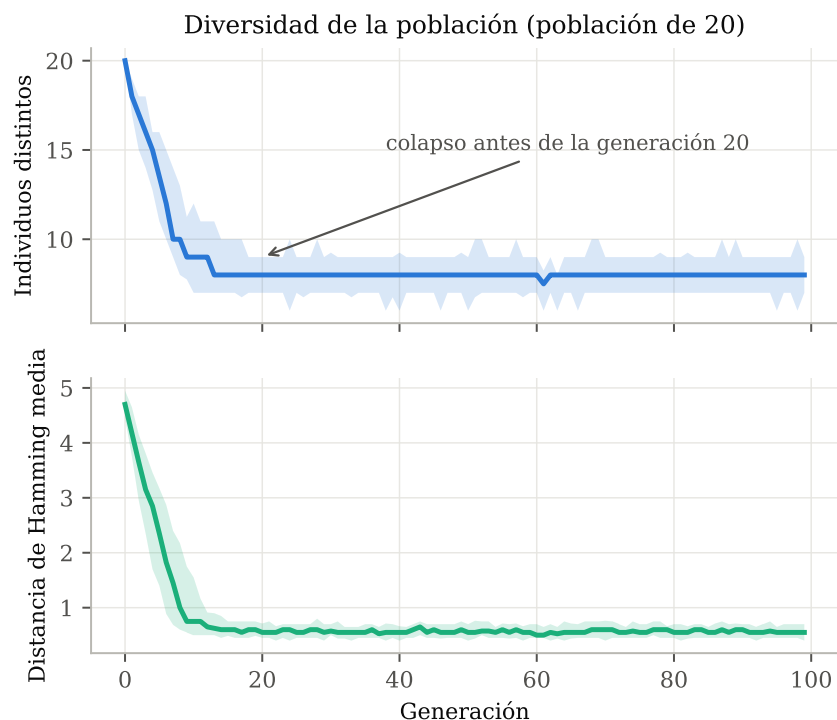


Figura 4: Colapso de la diversidad poblacional (mediana y rango intercuartílico sobre 100 semillas, $N = 20$, $p_m = 0,05$).

espacio de 1024 soluciones con un óptimo único y un paisaje plano, remuestrear un vecindario es estrictamente peor que muestrear al azar.

7 Respuestas a las preguntas planteadas

¿Qué efecto tuvo el tamaño de la población? Positivo pero secundario y decreciente. A presupuesto igualado, pasar de $N = 10$ a $N = 50$ multiplica la tasa de éxito por entre 1,2 y 6 según la mutación, mientras que el salto entre $N = 20$ y $N = 50$ con $p_m = 0,10$ (0,88 frente a 0,90) queda dentro del ruido. Sin igualar el presupuesto, el efecto aparente de la población es mucho mayor, pero se confunde con el cómputo adicional.

¿Cómo afectó la tasa de mutación a la diversidad? Es el factor dominante. Con $p_m = 0,01$ (0,1 genes mutados por individuo) la diversidad colapsa y la búsqueda se detiene: los éxitos provienen de la población inicial, no de la evolución. Con $p_m = 0,10$ (1 gen por individuo) la diversidad se sostiene lo bastante para alcanzar la línea base.

¿Una tasa de mutación alta produjo siempre mejores resultados? En el rango evaluado, sí, de forma monótona. Pero el resultado no se extrapola: la mejora proviene de que la mutación alta contrarresta la convergencia prematura, y su límite es la búsqueda puramente aleatoria, que en esta instancia acierta el 85,8% con 2000 evaluaciones. Una tasa de mutación creciente hace converger el algoritmo hacia esa cota, no por encima de ella.

¿El mejor individuo apareció antes de la última generación? Casi siempre. Con $p_m = 0,10$ y $N = 20$ la mediana de la generación del primer acierto es 24,5 sobre 100 generaciones; con $N = 50$ es 9,5 sobre 40. Las generaciones posteriores no aportan: la diversidad ya ha colapsado.

¿El algoritmo obtuvo siempre la misma solución? No. La solución devuelta depende de la semilla y del valor de λ . Con $\lambda = 5$ el atractor es el óptimo en 31 de 100 ejecuciones y óptimos locales a distancia de Hamming 3, 5 y 6 en el resto. El beneficio, en cambio, varía poco (95 a 100), consecuencia directa de la planitud del paisaje descrita en la Sección 2: el algoritmo casi siempre queda cerca y casi nunca acierta.

¿Qué evidencia indicaría convergencia prematura? Tres señales, todas medidas aquí: el colapso del número de individuos distintos (de 19,9 a menos de 9 antes de la generación 10); la caída de la distancia de Hamming media al mejor individuo por debajo de 1; y el estancamiento de la mejor aptitud mientras la aptitud promedio converge hacia ella, visible en la Figura 5. La coincidencia de ambas curvas es el síntoma clásico: la población ha dejado de explorar.

¿Qué configuración se recomendaría y por qué? Ninguna configuración del algoritmo genético es recomendable para este problema. La configuración C alcanza el 100% de éxito, pero la búsqueda aleatoria con sus mismas 10 000 evaluaciones también, y la heurística voraz obtiene el óptimo exacto en diez operaciones. Si aun así debe ejecutarse un algoritmo genético, la mejor configuración evaluada es $N = 50$, $p_m = 0,10$, λ algo por encima de 1,9 — y su mérito es igualar al azar, no superarlo.

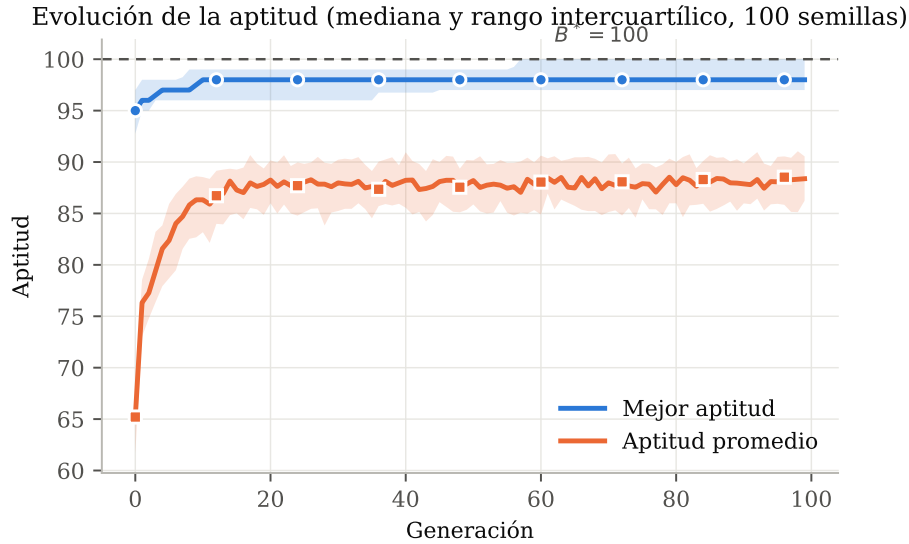


Figura 5: Evolución de la mejor aptitud y de la aptitud promedio. Se agregan las 100 ejecuciones (mediana y rango intercuartílico) en lugar de mostrar una trayectoria aislada, que en un algoritmo estocástico sería anecdótica.

8 Conclusiones

El algoritmo genético implementado funciona: converge, respeta la restricción con una penalización adecuada y alcanza el óptimo global con frecuencia creciente al aumentar la diversidad. El problema no es el algoritmo, sino su adecuación al problema. En esta instancia no supera a la búsqueda aleatoria en ninguna configuración, y la heurística voraz la resuelve de forma exacta en diez operaciones. Las condiciones que explican este resultado son identificables: espacio pequeño (2^{10}), óptimo único, paisaje plano por razones b_i/c_i casi idénticas, y una estructura que una regla voraz explota directamente.

Queda entonces la pregunta que sí tiene interés: cuándo estaría justificado un algoritmo evolutivo.

1. **Cuando el espacio sea inabordable por enumeración.** Con $n = 50$ proyectos el espacio tiene $2^{50} \approx 10^{15}$ soluciones y la búsqueda aleatoria deja de ser competitiva, porque su probabilidad de éxito — ecuación (4) — decae con $|\Omega|$ mientras que la de un algoritmo poblacional depende de la estructura, no del tamaño.
2. **Cuando ninguna heurística simple explote la estructura.** Aquí el voraz por razón funciona porque todos los proyectos tienen eficiencia parecida y el presupuesto se satura exactamente. Con costos y beneficios irregulares, o con presupuesto que no se satura, esa garantía desaparece.
3. **Cuando haya múltiples restricciones acopladas** (mochila multidimensional, precedencias entre proyectos, exclusiones mutuas), donde no existe una razón escalar que ordenar y la programación dinámica deja de ser pseudopolinómica en la práctica.
4. **Cuando la función objetivo sea costosa o no tenga forma analítica**, situación en que lo relevante es el número de evaluaciones y no el tamaño del espacio.

Ninguna de las cuatro condiciones se cumple en la instancia propuesta. Esa, y no el ajuste de parámetros, es la razón por la que el algoritmo genético no rinde aquí.

A Desviaciones respecto del enunciado

Generador aleatorio local. Se emplea una instancia `random.Random(semilla)` por ejecución en lugar de `random.seed(42)` global. Con 2200 ejecuciones, un estado global compartido hace que cualquier código que consuma números aleatorios altere la secuencia y se pierda la reproducibilidad sin aviso.

Semilla de la traza manual. Se usa la semilla 26 en lugar de 42, por las razones y con los criterios expuestos en la Sección 4.1.

Diseño experimental ampliado. Se conservan las configuraciones A, B y C del enunciado y se añaden el factorial a presupuesto igualado, el barrido de λ y la línea base, por los motivos de la Sección 5.

Dependencias. Solo `random`, `time` y `matplotlib`. El intervalo de Wilson, la prueba exacta de McNemar y la corrección de Holm se implementan sin bibliotecas externas.